Assignment 3 Report

For this assignment I wrote the same data cleaning pipeline twice, once in PySpark and once in Ray Data, and ran each one on its own cluster with two workers. The data was the NYC Yellow Taxi trip records. I timed every stage of both pipelines to answer a simple question: which framework cleans this data faster, and what does a custom Python function cost in each. The code is in my GitHub repository at https://github.com/Rishitha470/DA3408_Assignment3, in the files spark_clean.py and ray_clean.py.

1. Data and pipeline

I used 23 monthly parquet files of Yellow Taxi trips, from February 2023 to December 2024, together with the taxi zone lookup CSV. That comes to 76,413,180 rows before any cleaning. Both scripts take the same four arguments (trips path, zones path, output path and metrics path) and run the same five steps, each one timed separately.

Ingestion reads every parquet file and the zone CSV and counts the rows. Cleansing keeps six columns (pickup time, dropoff time, trip distance, pickup zone ID, dropoff zone ID and total amount), drops rows with nulls, drops duplicates and converts the two time columns to timestamps. The join attaches a zone name to both the pickup and the dropoff location. The UDF step computes avg_speed_mph for every trip as distance divided by trip duration in hours, and returns null when the duration is zero or negative. Export writes the result as parquet.

Two things had to be done differently because of the frameworks. Spark has a normal join, so I used it. Ray Data had no join operator I could use here, so I loaded the zone table into a pandas DataFrame and merged it inside map_batches. That works because the zone table is tiny, so every batch can hold a full copy of it, a bit like a broadcast join. The custom function is a Python UDF in Spark and a pandas batch function in Ray.

2. Cluster setup

Both clusters were started with Docker. For Spark I used a standalone Spark 3.3.4 cluster with one master and two workers, each worker having one core and 1024 MiB of memory. My application was given both cores and 512 MiB per executor. Figure 1 shows the master UI with two alive workers, and Figure 2 shows the SparkClean application running on both of them.

```
docker compose up -d
```

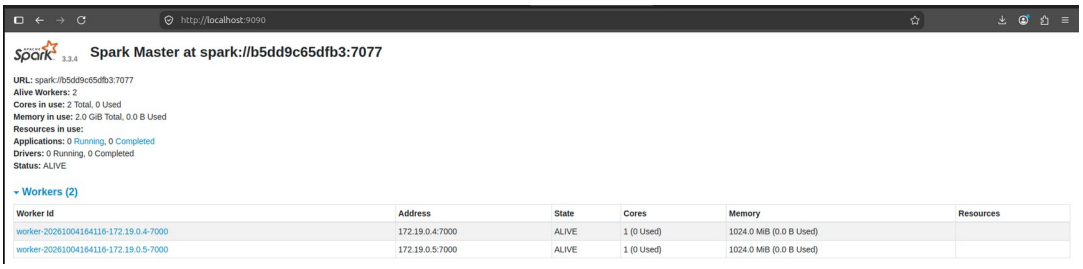


Figure 1. Spark Master UI on port 9090 with two alive workers.

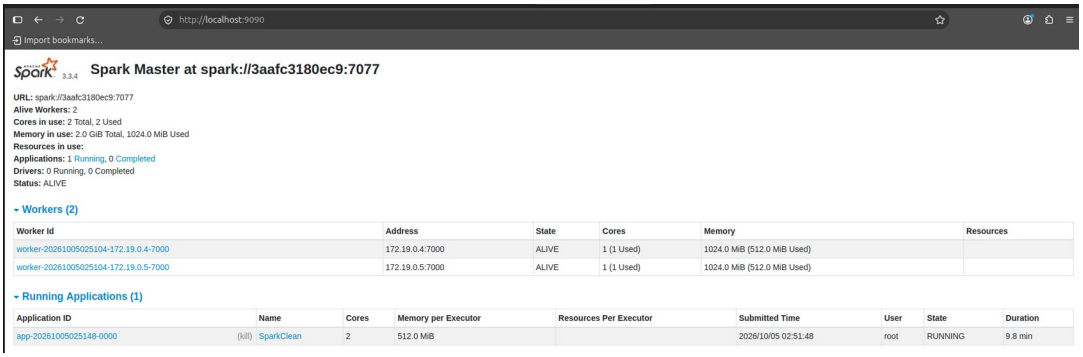


Figure 2. Spark Master UI with the SparkClean application running on both workers.

For Ray I used the 2.9.3 image with a head node and two workers, each container limited to 1 GB of memory, one CPU, a 200 MB object store and 512 MB of shared memory. Figure 3 shows the Jobs tab with ray_clean.py in the RUNNING state, and Figure 4 shows the cluster status during a run, with three active nodes and all 3.0 of 3.0 CPUs in use. Since the head node also offers a CPU, Ray had three CPU slots for work while Spark had two executor cores. I come back to this in the limitations.

```
docker compose -f ray-docker-compose.yml up -d
```

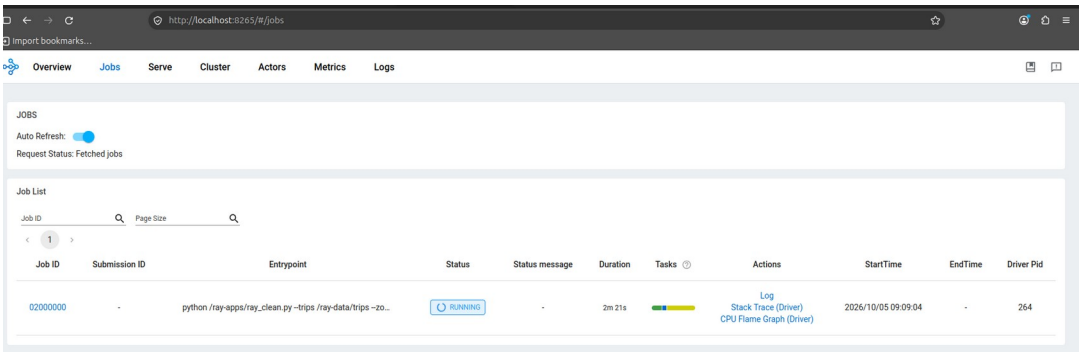


Figure 3. Ray Dashboard Jobs tab on port 8265 with ray_clean.py running.

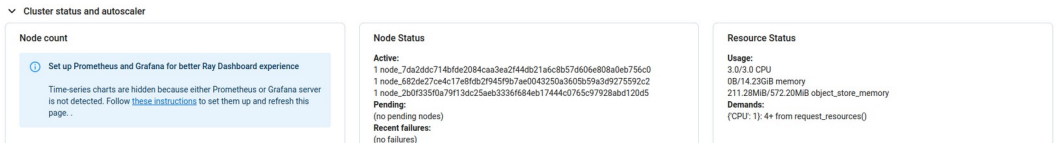


Figure 4. Ray Dashboard cluster status during a run: three active nodes and 3.0 of 3.0 CPUs in use.

3. Execution time

Table 1 and Figure 5 give the time for each stage, taken from the metrics files written by the two scripts. Ray finished the whole pipeline in 1294.19 seconds, about 21.6 minutes, and Spark took 1781.31 seconds, about 29.7 minutes. That makes Ray 1.38 times faster, or roughly 27 percent less time (487 seconds saved).

Table 1. Time per stage in seconds.

Stage	Spark (s)	Ray (s)	Ray speedup	Time saved (s)
Ingestion	21.33	18.74	1.14x	2.59
Cleansing	385.17	278.44	1.38x	106.73
Join	341.03	241.86	1.41x	99.17
Python UDF	315.43	263.61	1.20x	51.82
Export	714.77	491.54	1.45x	223.23
Total	**1781.31**	**1294.19**	**1.38x**	**487.12**

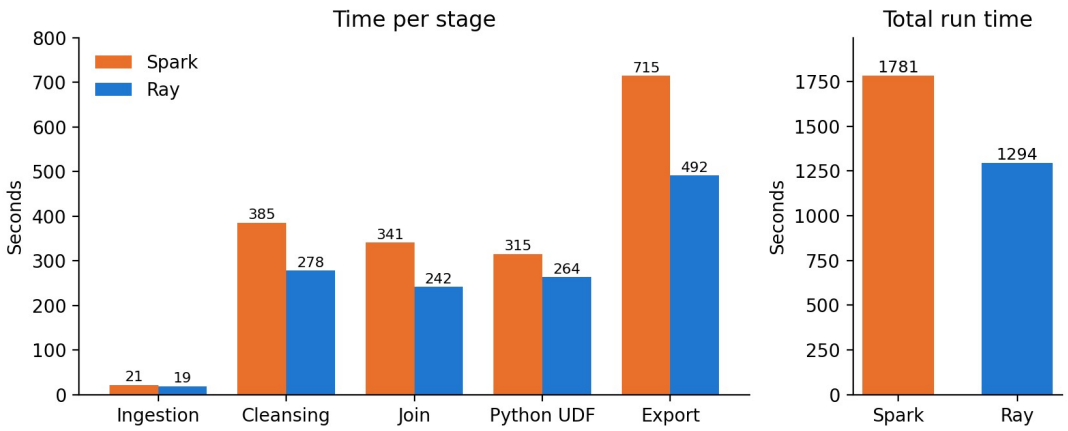


Figure 5. Time per stage and total run time, Spark against Ray.

Ray was ahead in every stage. The biggest gap in seconds was export, where Ray saved 223 seconds, and export was also the longest stage for both frameworks, taking about 40 percent of Spark's total and 38 percent of Ray's. Ingestion was nearly the same (21.33 against 18.74 seconds), and with only one run of each I would not read much into a difference of under three seconds.

The row counts matched exactly at every stage, as Table 2 shows. Cleaning removed 7 rows out of 76,413,180 in both frameworks, and the join and the UDF kept every row that was left.

Table 2. Row counts after each stage.

Stage	Spark	Ray	Result
Raw	76,413,180	76,413,180	Match
After cleaning	76,413,173	76,413,173	Match
After join	76,413,173	76,413,173	Match
After UDF	76,413,173	76,413,173	Match

4. Resource utilization

I logged CPU and memory with docker stats every few seconds while each job ran. The Ray log is complete and Table 3 summarizes it. The log from my Spark run only contains timestamps and no readings, so I do not have peak CPU or memory for Spark, and I left those cells empty rather than guess. The Spark UI does show that both executors were fully used, with 2 of 2 cores and 512 MiB per executor in use (Figure 2).

Table 3. Peak and average usage per container from docker stats.

Container	Peak CPU (%)	Average CPU (%)	Peak memory (MiB)	Average memory (MiB)
Ray head	316.4	59.8	975.2	303.8
Ray worker A	118.9	36.8	450.6	260.8
Ray worker B	110.8	24.5	786.8	267.4
Spark containers	Not captured	Not captured	Not captured	Not captured

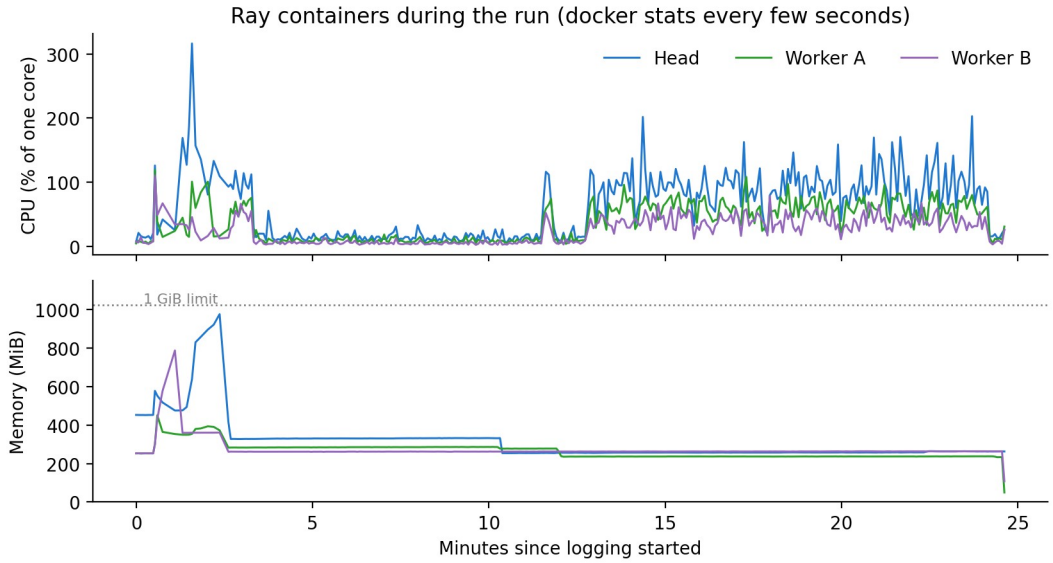


Figure 6. CPU and memory of the three Ray containers during the run, from docker stats.

docker stats reports CPU as a percentage of one core, so values above 100 mean a container used more than one core of the host. Ray's CPU setting only controls how many tasks it schedules and does not cap the container, which is why the head reached 316 percent. The head was also the hungriest container because the driver script runs inside it. It peaked at 975 MiB, about 95 percent of its 1 GiB limit, so memory was tight on that node. In Figure 6 the containers were busy in the first few minutes and again for the last twelve or so, with a quiet stretch in between where all three sat near 10 percent CPU. I did not dig into what caused that quiet stretch.

5. Python UDF overhead

In my runs the UDF stage took 315.43 seconds in Spark and 263.61 seconds in Ray, so Ray was 1.20 times faster. The reason I would expect a gap is how each framework runs Python. Spark executors run on the JVM, so a Python UDF has to send rows from the JVM to a separate Python worker process, run the function, and send the results back, with serialization at both ends. The function in my Spark script works on one row at a time. In Ray the pipeline is already Python, so the function runs in the same process that holds the batch and works on a whole pandas batch at once with vectorized operations. There is no language boundary and no call per row.

I want to be careful about how much my numbers prove, though. The UDF stage is a timed count after the UDF is applied, and none of the stages cache their input, so each stage time can also include upstream work that had to be redone. That means the stage times are not clean costs of one operation. Figure 7 shows an example: job 4 in Spark finished in half a second because it skipped stages whose

shuffle output already existed. This is also a likely reason why the Spark UDF stage (315 seconds) came out shorter than its cleansing stage (385 seconds) even though it covers more work. Ray Data is lazy too, so I expect it repeats work between counts in a similar way, but I did not check that in the Ray logs.

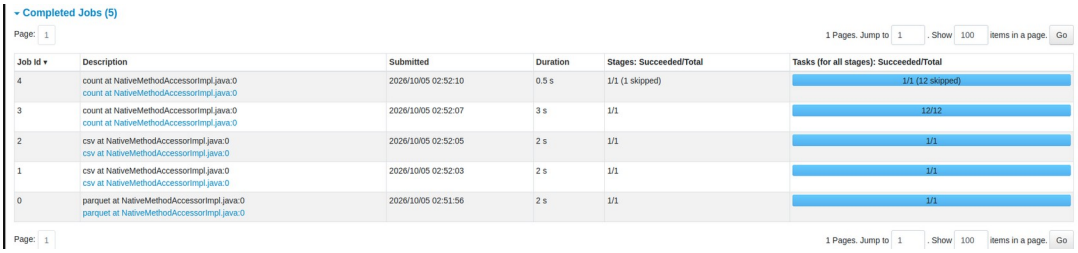


Figure 7. Spark application UI on port 4040, completed jobs. Job 4 skipped 12 tasks and finished in 0.5 seconds.

The 1.20 times gap for the UDF stage was also smaller than the gaps for cleansing (1.38), join (1.41) and export (1.45). So on this setup the Python boundary does not look like the main reason Spark was slower, and part of Ray's lead could come from its extra CPU slot or from my Spark function being row based instead of vectorized. I did not try a Spark pandas UDF or a version written with Spark's own functions, so I cannot say how much of the gap would disappear. That is the first thing I would test next.

6. Limitations

Four things limit how far these results go. First, the clusters are not the same size: Ray had three CPU slots including the head while Spark had two executor cores, and Spark executors had 512 MiB while each Ray container had 1 GB. Second, Ray's drop_duplicates runs inside each batch, so it only removes duplicates within a batch, while Spark's dropDuplicates compares all rows. The counts matched here, but a strictly identical pipeline would need a global duplicate removal in Ray. Third, I ran each pipeline once, so I have no spread to report. Fourth, the missing Spark resource log makes the resource comparison one sided.

7. Conclusion

Winner: Ray. It finished the pipeline in 1294 seconds against 1781 for Spark, it was faster in every stage, and the row counts matched at every step. Given the limitations above, I would treat this as a result for this small two worker setup and not as a general rule.

For an AI first project I would choose Ray. The pipeline is plain Python and pandas, custom functions need no extra serialization step, and the data stays in the same Python ecosystem that the training code uses, which matters when preprocessing feeds straight into model training. For a BI first project I would choose Spark. It has real SQL and DataFrame joins that work at any size, an optimizer, operations like duplicate removal that behave exactly as written across the whole dataset, and much wider support from BI tools and warehouses. In my own pipeline the Ray join was a hand written merge that only works because one table was small, and that is the kind of shortcut that would break on a heavier join.